Programmation Parallèle

Fiche 5 : Cache

Cette fiche ainsi que les fichiers à télécharger sont disponibles sur le site
https://gforgeron.gitlab.io/pap/

L'objectif est de mettre en œuvre des techniques d'optimisation pour éviter les défauts de cache inutiles. Pour simplifier les expériences nous allons supprimer les optimisations du compilateur. Insérer les lignes suivantes dans le fichier `CMakeLists.txt` à la ligne 33 :

```
set(CMAKE_C_FLAGS_RELEASE "-O0 -DNDEBUG" CACHE STRING "" FORCE)
set(CMAKE_CXX_FLAGS_RELEASE "-O0 -DNDEBUG" CACHE STRING "" FORCE)
```

Spécifités de la salle 104

Un point important est que les processeurs de la salle 104 sont hétérogènes. On peut travailler uniquement les gros cœurs, uniquement les petits ou bien une combinaison des deux. Cela se fait à l'aide de la variable `OMP_PLACES` :

```
OMP_PLACES=cores # toute la machine
OMP_PLACES=cores OMP_NUM_THREADS=8 # les 8 gros cœurs
OMP_PLACES={8}:16:1 OMP_NUM_THREADS=16 # les 16 petits cœurs
```

Scrollup

Nous allons modifier le noyau `scrollup` du programme EasyPAP afin provoquer de nombreux défauts de cache et observer leur impact sur les performances.

1. Produire une version `scrollup_compute_ji()` en dupliquant la fonction `compute_seq()` :

```
for (int i = 0; i < DIM; i++) {
  int src = (i < DIM - 1) ? i + 1 : 0;
  for (int j = 0; j < DIM; j++)
    next_img (i, j) = cur_img (src, j);
}
```

   et en inversant l'ordre des boucles en `i` et `j`, on pourra aussi éliminer le test en traitant à part le cas de la dernière ligne :

```
for (int j = 0; j < DIM; j++)
     next_img (DIM - 1, j) = cur_img (0, j);
```

2. En utilisant l'image `shibuya.png`, vérifier visuellement le résultat. Comparer « visuellement » la fluidité de l'animation obtenues par les versions `seq` et `ji` du noyau `scrollup`.

3. Utiliser le script `plots/run-xp-scrollup.py` pour lancer une série d'expériences qui va faire varier la taille de l'image et le nombre d'itérations pour travailler à coût constant :

$$\text{itérations} = \frac{4096 \times 4096}{\text{DIM} \times \text{DIM}}$$

$$\text{volume de calcul} = \text{itérations} \times \text{DIM}^2 = \frac{4096 \times 4096}{\text{DIM}^2} \times \text{DIM}^2$$

4. Utiliser la commande suivante pour générer un élégant graphique :

```
./plots/easyplot.py  -y time -x size --yscale log --xscale log2 \
                     --delete iterations -if scrollup.csv -- aspect=2
```

5. Interpréter ce graphique sachant qu'un pixel est codé sur 4 octets et que les tailles des caches de la machine peuvent être obtenues via la commande `lstopo`.

Remettre le Makefile dans son état initial.

Multiplication de matrices

Le programme `mul_mat.c` effectue une multiplications de matrices de façon classique.

1. Modifier le code de `mulMat2` afin d'utiliser plus efficacement le cache du processeur. Le gain obtenu est-il décevant, correct ou plus que satisfaisant ?
2. Il est probable que quelques défauts de cache évitables subsistent. Les repérez-vous ? Quelle permutation des boucles sur i, j, k induit le plus petit nombre de défauts de cache ? Modifier votre code en conséquence.

Bonus Lorsque N est assez grand il est probable quelques défauts de cache évitables subsistent dans votre code. Supposons que le cache fasse 8 Mo pour quelle valeur de N apparaissent ces défauts de cache ? Quelle technique faudrait-il utiliser pour limiter ces défauts ?

Rotation d'une image

Nous allons étudier l'impact du cache sur les performances du noyau `rotation90` du programme EasyPAP.

1. Produire une version optimisée `rotation90_compute_tiled()` en utilisant la technique de pavage (tuilage). On s'inspirera du code de la fonction `mandel_compute_tiled` pour cela.
2. Comparer les performances obtenues.
3. Produire une version parallèle de chaque code (`omp` et `omp_tiled`), comparer les performances obtenues pour une image de taille `2048`.
4. Utiliser le script `plots/run-xp-rotation90.py` pour vos expériences et produire ainsi un graphique.

Impact du placement mémoire sur les architectures NUMA

Les serveurs du CREMI (joliecoeur, ombrage, tesla) sont des machines multiprocesseur, les processeurs étant reliés par un réseau. Les cœurs de chaque processeur accèdent directement à des bancs mémoire locaux et doivent passer par le réseau pour accéder aux bancs distants. La latence mémoire est fonction de la distance réseau entre le cœur et le banc mémoire de la donnée.

En vous aidant de la fonction `touch_tile ()`, écrire une fonction **`void`** `rotation90_ft ()` *first touch* pour faire en sorte que les pages mémoire traitées soient distribuées sur la machine.

```
1 static void touch_tile (int x, int y, int width, int height)
2 {
3
4   for (int i = y; i < y + height; i++)
5     for (int j = x; j < x + width; j++)
6       next_img (i, j) = cur_img (j, i) = next_img (j, i) = cur_img (i, j) = 0;
7 }
```

Ajouter les lignes suivantes au fichier `plots/run-xp-rotation90.py` et relancer les expériences pour ajouter de nouvelles données au fichier `rotation90.csv`.

```
easyspap_options["-ft"] = [""]
easyspap_options["--label"] = ["with-ft"]
```